CS6650 Final Project Plan

Scaling a Decentralized Social Network: Stress-Testing Mastodon's Distributed Architecture

Team: Yaoyi Wang & Yehe Yan **Presentation:** Apr 13 / 20 **Final Deadline:** Apr 20

---

1. Project Goal

This project studies how a federated social network behaves under load by using Mastodon as a real-world case study.

Our original goal was to deploy Mastodon on AWS using a production-style architecture and measure its behavior under load. After Week 1 investigation, we found that the AWS Academy Learner Lab environment imposes significant IAM-related restrictions that block the original ECS/Fargate + CloudFormation deployment path.

As a result, we are pivoting to a **tiny Mastodon deployment strategy** based on **EC2 + Docker Compose**. This allows us to preserve the core research goals of the project while increasing the probability of delivering a working system and meaningful performance observations.

Our project now focuses on three questions:

1. **Single-instance bottleneck** — which component becomes the bottleneck first under load?
2. **Bottleneck shifting / component dependency** — if web-side capacity is increased or system protections are relaxed, does the bottleneck move toward PostgreSQL or other internal components?
3. **Federation feasibility and propagation behavior** — can two lightweight EC2-hosted Mastodon instances support a basic federated workflow, and what happens to cross-instance delivery under simple failure or latency tests?

---

2. Success Criteria

Must-have

- Two working Mastodon instances on AWS EC2
- Valid Locust smoke test
- Single-instance bottleneck experiment with data
- Basic federation validation with screenshots / observations
- At least 2–3 result graphs for the presentation
- Clear documentation of deployment tradeoffs and system limitations

Nice-to-have

- Bottleneck-shifting or worker-scaling observations
- Cache / queue / DB connection dependency observations
- A small federation latency or partial-outage test

Stretch

- Sidekiq queue backlog analysis
- Retry analysis
- Worker/process tuning observations
- Comparative notes between the original CloudFormation path and the final EC2-based path

---

3. Go / No-Go Checkpoints

Checkpoint 1 — Mar 22 If the original CloudFormation/Fargate path remains blocked by Learner Lab IAM restrictions, pivot to EC2 + Docker Compose.

Checkpoint 2 — Mar 30 If the second instance or federation path is still unstable by Mar 30, federation will be reduced to:

- architecture discussion
- expected propagation path
- future work section

---

4. Risk Register

Risk	Impact	Mitigation
Learner Lab IAM blocks CloudFormation/Fargate resources	Original deployment path fails	Pivot to EC2 + Docker Compose
EC2-based deployment still takes too long to stabilize	Delays experiments	Reduce to one instance first and prioritize bottleneck testing
Federation setup unstable	Week 3 slips	Keep federation as optional / future work
Manual deployment introduces configuration drift	Weak reproducibility	Record all setup steps carefully in repo notes
4-hour lab timeout or environment interruptions	Lost progress	Save screenshots, commit notes frequently, stop/restart resources carefully

---

5. AWS Academy Learner Lab Constraints

Constraint	Notes
Session timeout	Each session expires after about 4 hours
Region restrictions	Only `us-east-1` or `us-west-2` available
IAM restrictions	Limited permissions; blocked CloudFormation/ECS task role creation
Budget cap	Limited credits per account

Constraint	Notes
Resource recovery	Some resources may need manual restart after timeout

Operational habits

- Use one agreed region consistently (`us-east-1` unless blocked)
- Stop non-essential resources after each session
- Save screenshots and deployment notes after every major step
- Expect to restart or recover resources after session timeout

---

6. Deployment Strategy

Original path (investigated in Week 1)

We initially attempted to deploy Mastodon using a modified version of the `widdix/mastodon-on-aws` CloudFormation stack.

This path was progressively simplified:

- remove SES
- remove CloudFront
- remove S3
- remove VPC Flow Logs
- remove Route 53 / ACM-managed resources
- switch to HTTP only

Despite these changes, deployment still failed because Learner Lab blocked creation of ECS task-related IAM roles (`TaskRole`, `TaskExecutionRole`).

New path (selected after Week 1)

We are pivoting to a **tiny Mastodon deployment** using:

- **EC2**
- **Docker Compose**
- **minimal Mastodon runtime components**

This strategy reduces infrastructure complexity while preserving the ability to:

- run Mastodon
- generate load
- observe queue behavior
- measure latency and throughput
- attempt federation across two simplified instances

Current plan

- Each teammate attempts one Mastodon instance using the simplified EC2-based deployment path
- `main` keeps shared structure, scripts, and documentation

- Federation is attempted later only if both instances become stable

---

7. Team Responsibilities

Area	Owner	Instance	Main Focus
Single-instance bottleneck / bottleneck shifting	Yaoyi	Instance A	Load generation, latency analysis, web-layer bottleneck observations, result formatting
Component dependency / backend behavior	Yehe	Instance B	Redis / Sidekiq / DB connection observations, queue behavior, backend interpretation
Federation experiments	Shared	A + B	Cross-instance discovery, propagation, and simple failure / latency tests
Shared repo structure / scripts	Shared	-	Locust scripts, result format, report integration

---

8. Metrics

Tier 1 (must-have)

- Request latency (P50 / P95 / P99)
- Throughput (RPS)
- Error rate
- CPU / memory utilization on EC2 or container host
- Database behavior (connections / CPU if observable)
- Queue / worker backlog indicators if observable

Tier 2 (best effort)

- Redis queue depth
- Sidekiq retries
- Worker-specific processing time
- Federation propagation latency under different load levels

---

9. Repository Structure

```
mastodon-scaling-study/
├── README.md
├── cloudformation/
│   ├── v1-no-ses.yml
│   ├── v2-no-ses-no-cloudfront-s3-public.yml
│   ├── v3-no-ses-no-cloudfront-no-s3.yml
│   ├── v4-http-only.yml
│   └── v5-http-only-flowlog-false.yml
├── infra/
│   └── cloudwatch-dashboard.json
```

```
├── locust/
│   ├── locustfile.py
│   └── federation_test.py
├── report/
│   └── mastodon_plan.md
└── results/
    ├── yaoyi/
    │   ├── screenshots/
    │   └── week1_notes.md
    └── yehe/
```

---

10. Mastodon Architecture Overview

Original target architecture

```
Client Request
   ↓
ALB
   ↓
Web (Ruby on Rails / Puma)  ←→  Redis (queue + cache)
   ↓                                  ↓
PostgreSQL                      Sidekiq workers
                                      ↓
                             Federation HTTP POST → Remote Instance
```

Pivoted minimal architecture

```
Client Request
   ↓
EC2 host
   ↓
Docker Compose services
   ├── Mastodon web
   ├── Streaming API
   ├── Sidekiq
   ├── PostgreSQL
   └── Redis
```

Federation path

1. A user on Instance A posts a status
2. Sidekiq creates an ActivityPub::DeliveryWorker job
3. Instance A sends an HTTP POST to Instance B's /inbox
4. Instance B processes the activity through Sidekiq
5. The post appears in Instance B's timeline

---

11. Week 1 (Mar 20–23): Deployment Validation + Smoke Test Preparation

Primary objective

Determine whether Mastodon can be deployed in AWS Academy Learner Lab using the original CloudFormation-based path.

What was done

- Generated deployment secrets
- Purchased and configured domain
- Created Route 53 hosted zone
- Iteratively simplified the CloudFormation templates (v1–v5)
- Recorded failure points and screenshots

Key result

The CloudFormation/Fargate path is blocked by Learner Lab IAM restrictions, especially around ECS task role creation.

Week 1 deliverables

- Updated project plan
- Updated README
- Deployment failure analysis and screenshots
- Confirmed pivot decision to EC2 + Docker Compose

---

12. Week 2 (Mar 24–30): Minimal Deployment + Smoke Test

Yaoyi — Instance A

Goal: get one tiny Mastodon instance running on EC2 and prepare for load testing.

Tasks:

- Set up EC2-based Mastodon deployment
- Validate web access and basic application readiness
- Prepare Locust smoke test workflow
- Define result format for experiments

Yehe — Instance B

Goal: get a second tiny Mastodon instance running and focus on backend / queue observability.

Tasks:

- Set up EC2-based Mastodon deployment
- Document backend process / queue behavior
- Prepare federation prerequisites if time allows

Shared Week 2 outputs

- One working minimal instance
- Smoke test results
- Updated deployment notes
- Clear next-step plan for experiments

---

13. Week 3 (Mar 31–Apr 6): Core Experiments + Federation

Experiment 1 — Single-Instance Bottleneck Measure:

- latency
- throughput
- error rate
- CPU / memory behavior
- queue backlog indicators

Experiment 2 — Bottleneck Shifting / Component Dependency Focus on:

- whether increasing web-side capacity pushes the bottleneck toward PostgreSQL
- cache / Sidekiq / DB connection behavior under load
- practical service dependency observations in the tiny deployment model

Experiment 3 — Federation Validation / Simple Failure Test Focus on:

- cross-instance account discovery and interaction
- propagation of posts / likes across instances
- optional partial-outage test (temporary stop / restart of one instance) and catch-up behavior after recovery

---

14. Final Week: Analysis, Report, Presentation

Project Timeline

Date Range	Task
Apr 4–5	Finish deployment validation and collect experiment data
Apr 6–8	Complete report draft and merge stable work into main
Apr 9–11	Create slides with at least 2–3 result figures
Apr 12	Rehearsal
Apr 13 / 20	Presentation / final submission window

Planned report structure

1. Introduction & Motivation
2. Original Architecture Goal vs. Real Deployment Constraints
3. Week 1 Feasibility Investigation
4. Pivot to Tiny Mastodon on EC2

5. Experiment 1: Single-Instance Bottleneck
6. Experiment 2: Bottleneck Shifting / Component Dependency
7. Experiment 3: Federation Validation / Simple Failure Test
8. Discussion: deployment tradeoffs, bottlenecks, cache / queue / dependency behavior
9. Conclusion & Future Work

---

15. Cost Strategy

Because Learner Lab credits are limited, our strategy is to minimize runtime and avoid keeping all resources active simultaneously.

Item	Strategy
Week 1 investigation	fast validation, screenshot capture, short sessions
Week 2 deployment	minimal instance size and short validation cycles
Week 3 experiments	short headless Locust runs
Federation	only attempt if both stability and credits allow

We will stop non-essential resources after each session.

---

16. Tech Stack

Component	Tool
Deployment	EC2 + Docker Compose
Compute	EC2
Web / App	Mastodon v4.5.7
Database	PostgreSQL 14 (Docker)
Cache / Queue	Redis 7 + Sidekiq
Reverse Proxy / HTTPS	Nginx + Let's Encrypt
Monitoring	Basic AWS / system observation, Docker stats, screenshots, logs
Load Testing	Locust (Python)
Version Control	GitHub (mastodon-scaling-study)